Design and Comparative Evaluation of Different AI Algorithms for Mancala

Shuyang Huang

Matthew Kolker

Elliott Armstrong

Foundations of AI (*Spring, 2026*)



PennState

Context

- Mancala Game Framework
  - Focuses on Kalah variant - 48 seeds across 12 pits
- Core Mechanics
  - Sowing Logic
  - Extra Turn Rules
  - Capture Mechanics
- Heuristics
  - Uses guiding strategies to streamline the decision making process
- Search Optimization
  - Reduces node expansion to fit computational budges
- Strategic Objective
  - Evaluate board states to maximize the sum of stones in the goals while balancing immediate gains against future potential advantages



PennState

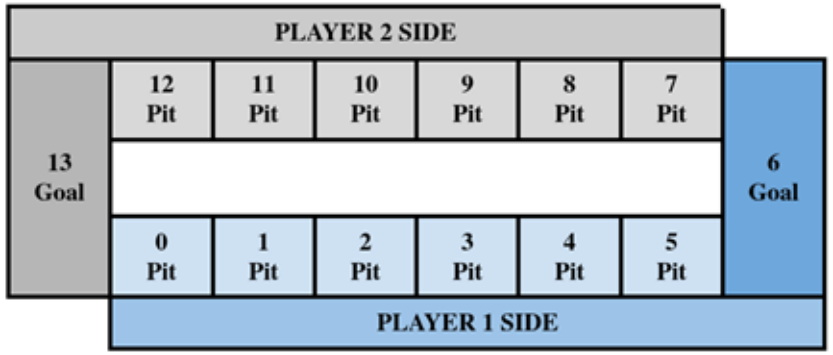
Problem

- Massive State Space
  - Stronger algorithms improve through different mechanisms:
    - Deeper lookahead, better pruning, or more simulations.
- Complex Rules
  - Extra-turn rules that break simple turn alternation
  - Capturing stones from opponents
  - Strong first-mover advantage.
- Objective
  - Identify the threshold of diminishing returns between AI agents
  - Establish the practical limits of specific algorithms in Mancala

PennState

Technical Approach

- Build a complete Mancala simulation Python environment
- Implement five AI approaches for comparison
  - Random
  - Greedy
  - Minimax
  - Alpha-Beta Pruning
  - Monte Carlo Tree Search (MCTS)
- Adjust algorithms for Mancala's extra-turn rule
- Evaluate both search depth and heuristic design
- Simulate tournaments with seat-swapping
- Compare strength, efficiency, and tactical behavior
  - Understand which agent wins and the performance of the models.

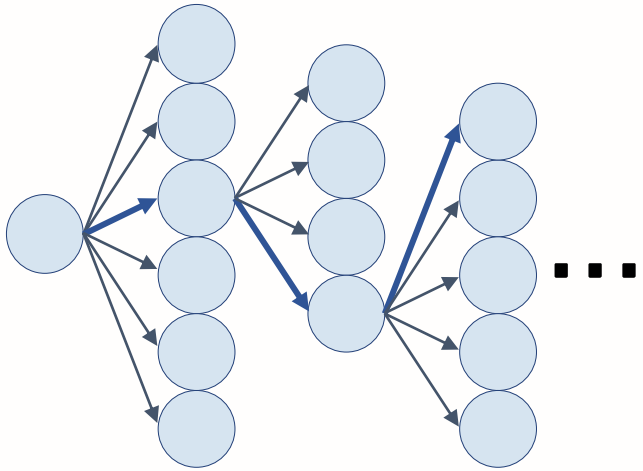


PennState

AI Model 1: Random Agent

- Selects a move at random from a list of available legal moves
  - Uses no search and no evaluation function
  - Worst expected performance
- Serves as a lower-bound baseline
  - Useful for showing how much value stronger algorithms add beyond chance play

Figure 1
Random Agent Diagram

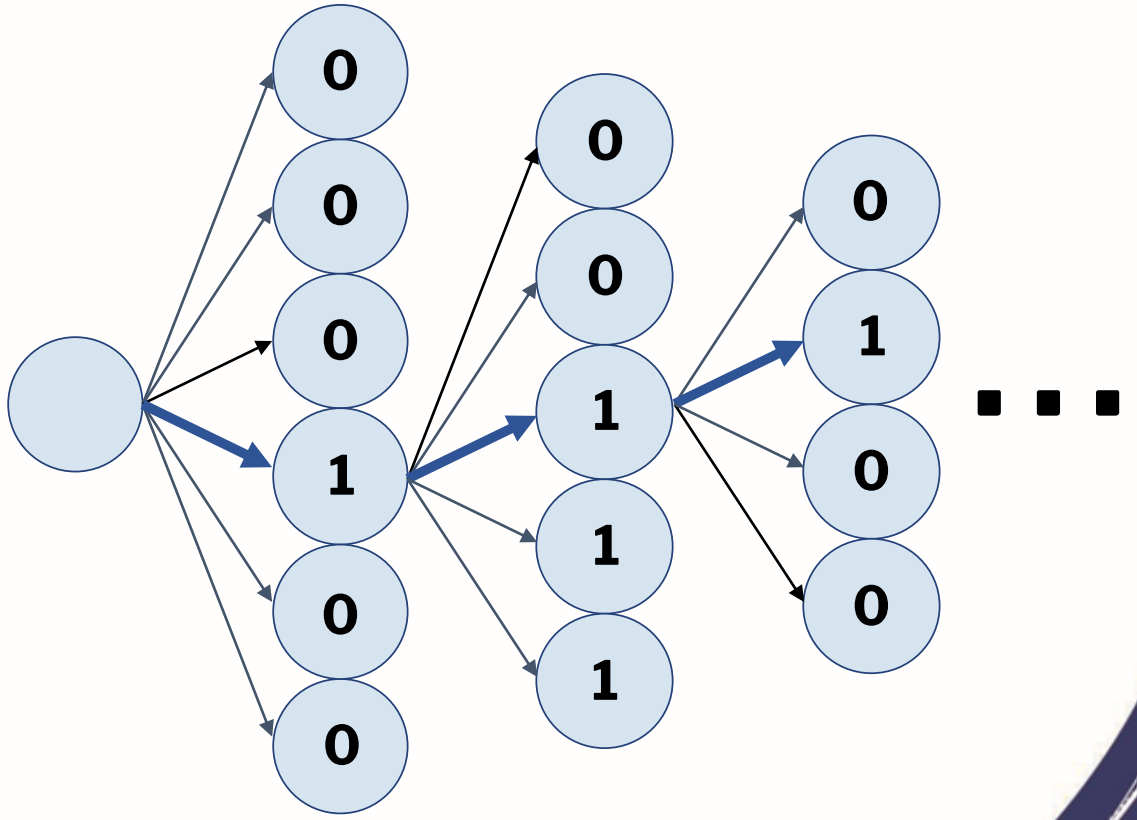


PennState

AI Model 2: Greedy Agent

- Chooses the move with the largest immediate store gain
  - Looks one move ahead only
    - Chooses randomly among any moves with an equal amount of gain
  - Does not model opponent responses or future consequences
- Serves as a stronger baseline than Random, but still very limited strategically

Figure 2
Greedy Agent Diagram

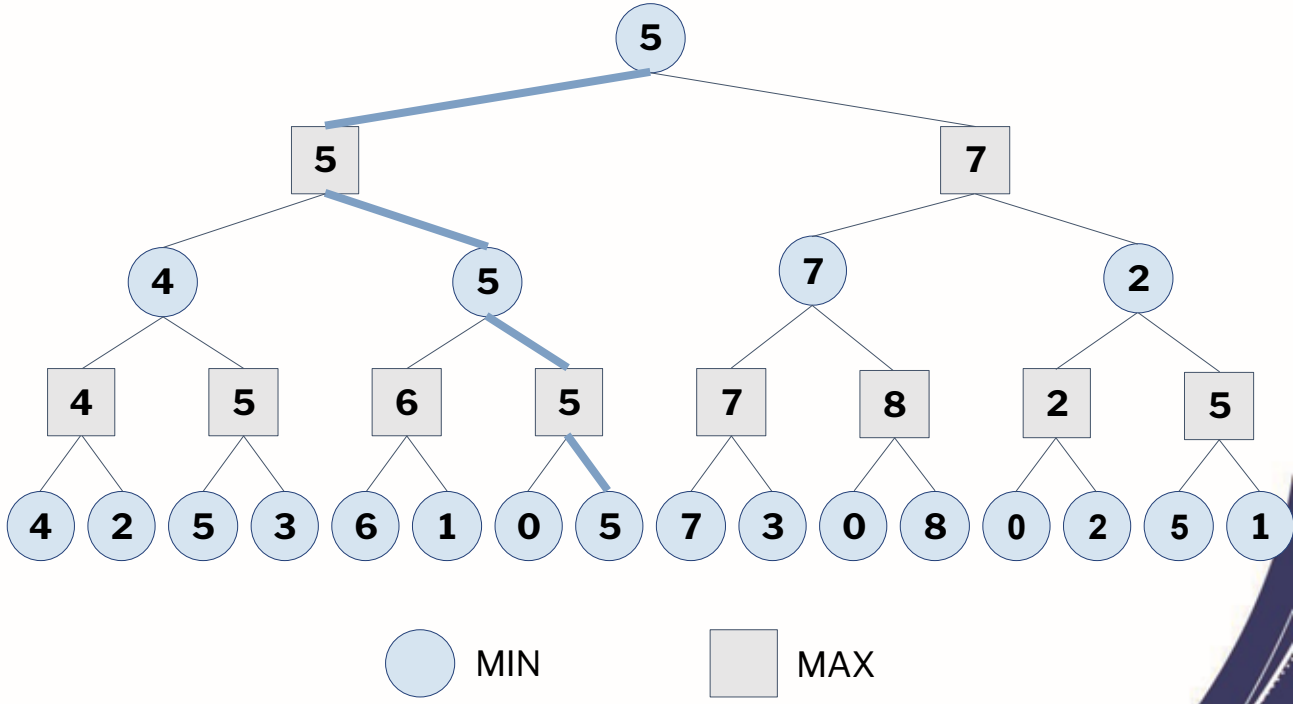


PennState

AI Model 3: Minimax

- Uses adversarial search to look ahead multiple moves
  - Assumes the opponent also plays optimally
- Chooses moves that maximize its own outcome while minimizing the opponent's
  - Uses a heuristic evaluation function at the depth cutoff
- Includes a depth extension for extra turns
  - Consecutive turns are handled correctly

Figure 3
Minimax Agent Diagram

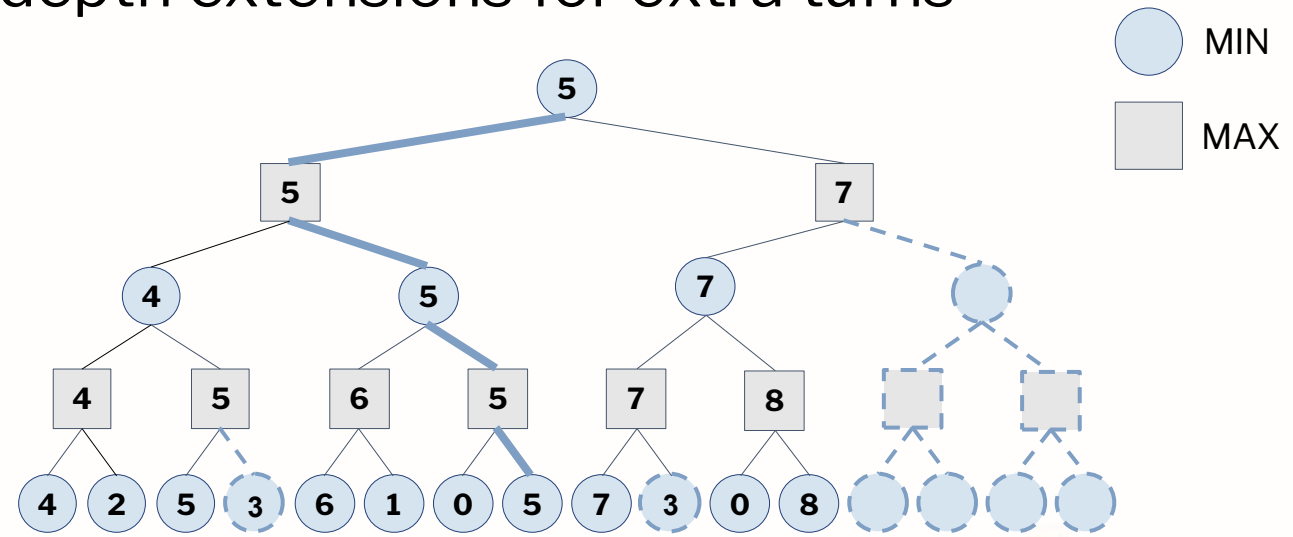


PennState

AI Model 4: Alpha-Beta Pruning

- Finds the same move as Minimax at the same depth, but searches faster
  - Skips branches that cannot affect the final decision
  - Reduces unnecessary node evaluation
- Uses move ordering to increase pruning effectiveness
  - Prioritizes extra-turn moves and immediate store gain
  - Also applies depth extensions for extra turns

Figure 4
Alpha-Beta Agent Diagram

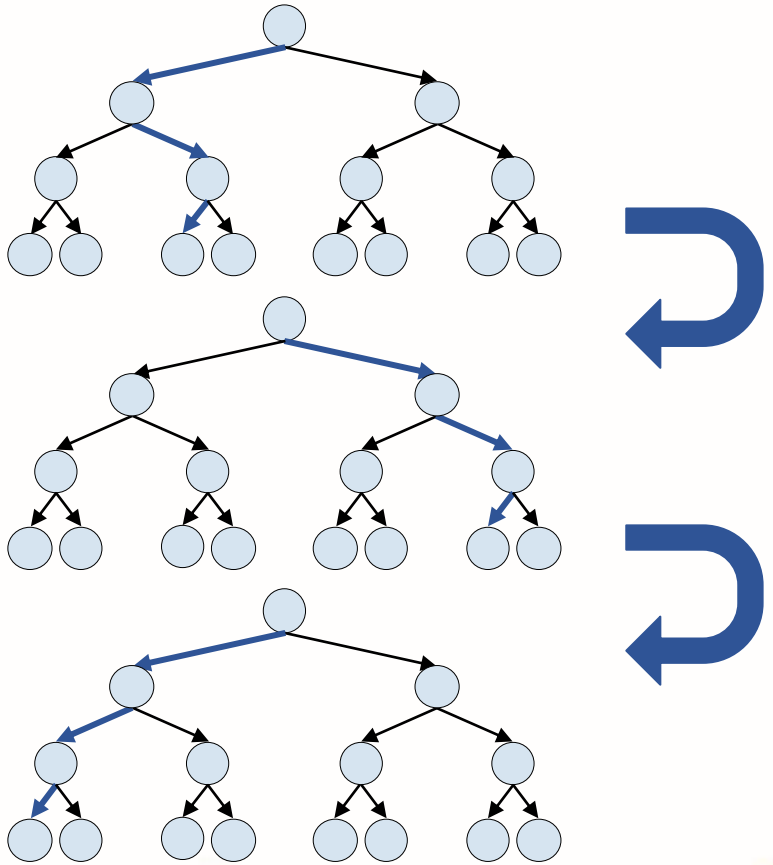


PennState

AI Model 5: Monte Carlo Tree Search

- Uses simulation-based search rather than a handcrafted heuristic
  - Performance scales with the simulation budget
  - Adapted for Mancala's non-alternating extra-turn structure
- Repeatedly runs rollouts from candidate positions to estimate move quality
  - Uses uniform confidence bounds for trees (UCT) to balance exploration and exploitation
- Useful for comparing simulation-based planning vs. depth-limited search

Figure 5
MCTS Diagram

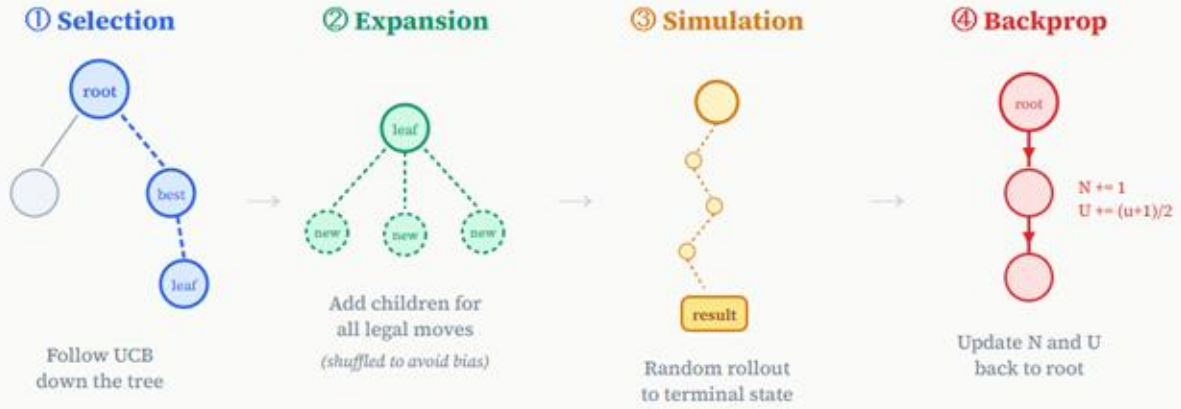


PennState

MCTS Iteration Cycle

MCTS Iteration Cycle — Four Phases

Repeated N times per move decision; each iteration refines the search tree



↻ **Repeat N times** *(N = simulation budget: 1,000 – 100,000 per move), then select the most-visited root child.*

1 Selection

Traverse tree using UCB1 to balance exploitation of strong moves with exploration of untested ones.

2 Expansion

Add all legal children of the reached leaf. Shuffle to avoid systematic pit-index bias.

3 Simulation

Random rollout to terminal state. Return game utility from root player's perspective.

4 Backprop

Update N and U back to root. Map utility: $\{-1,0,+1\} \rightarrow \{0, 0.5, 1\}$ so draws ≠ losses.

PennState

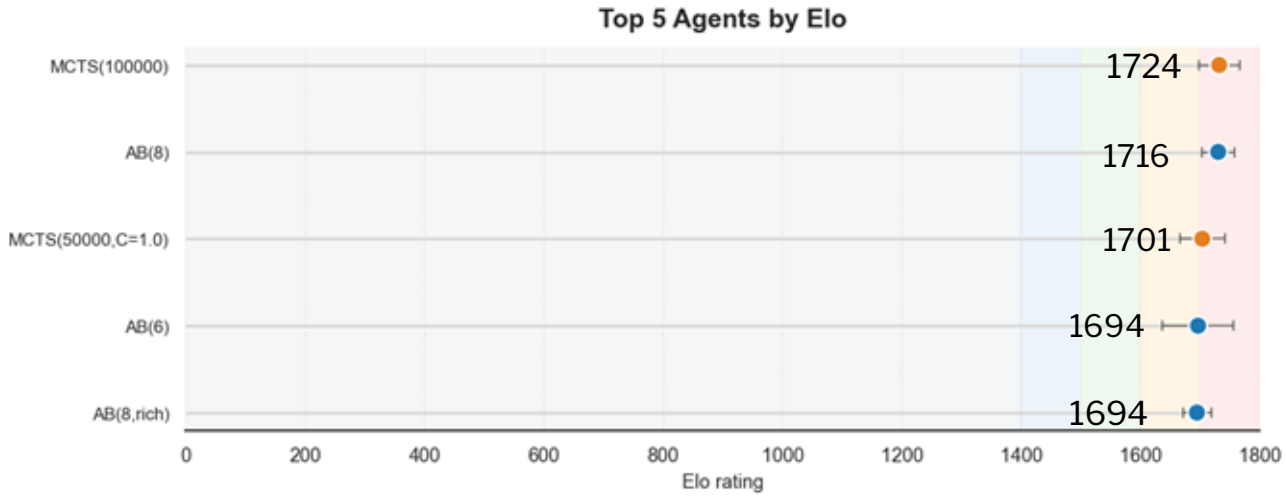
Evaluations – Metrics

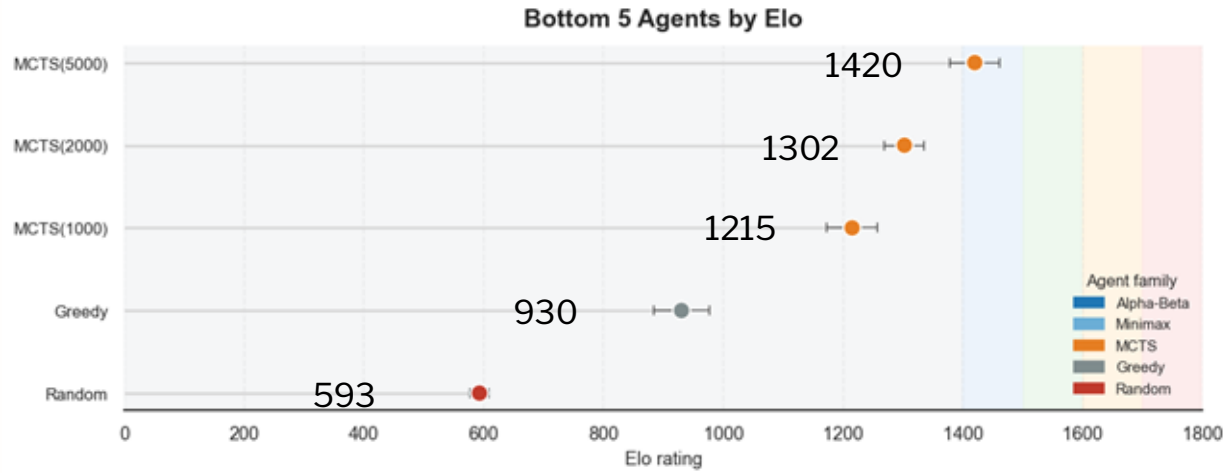
Item	Value
Total games	9,408
Matchup configurations	117
Agent configurations	30
Seat protocol	Equal games as Player 1 and Player 2
Timing normalization	Benchmark-calibrated across 2 machines
Win-rate uncertainty	95% Wilson confidence intervals
Significance test	Binomial test vs. 50% null
Effect size	Cohen’s h

PennState

Evaluations – Performance

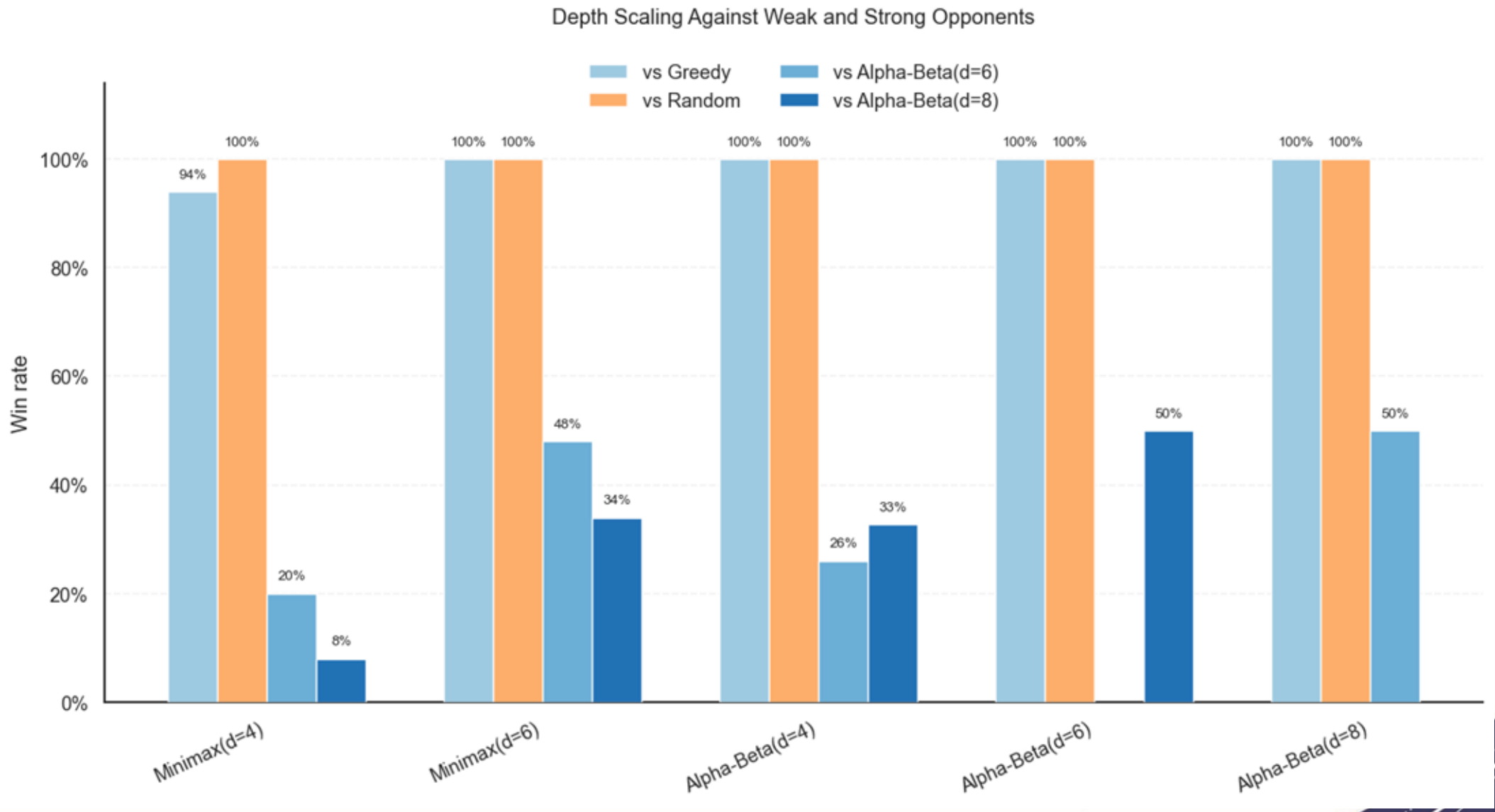
- Elo rating calculate relative skill
  - Predicts who should win
  - Gain points on wins
  - Deduct points on losses



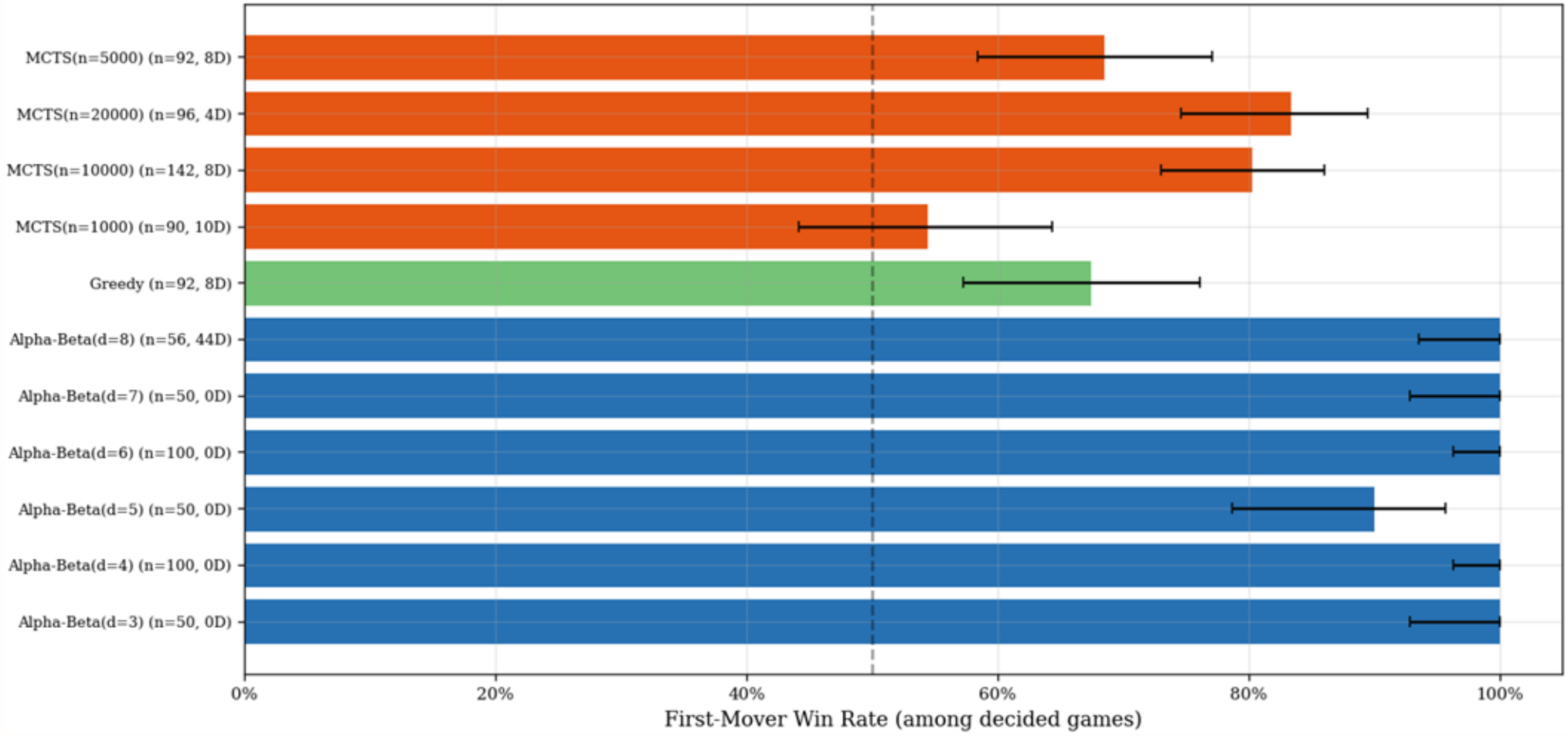


PennState

Effect of Search Depth



Evaluations – Performance



PennState

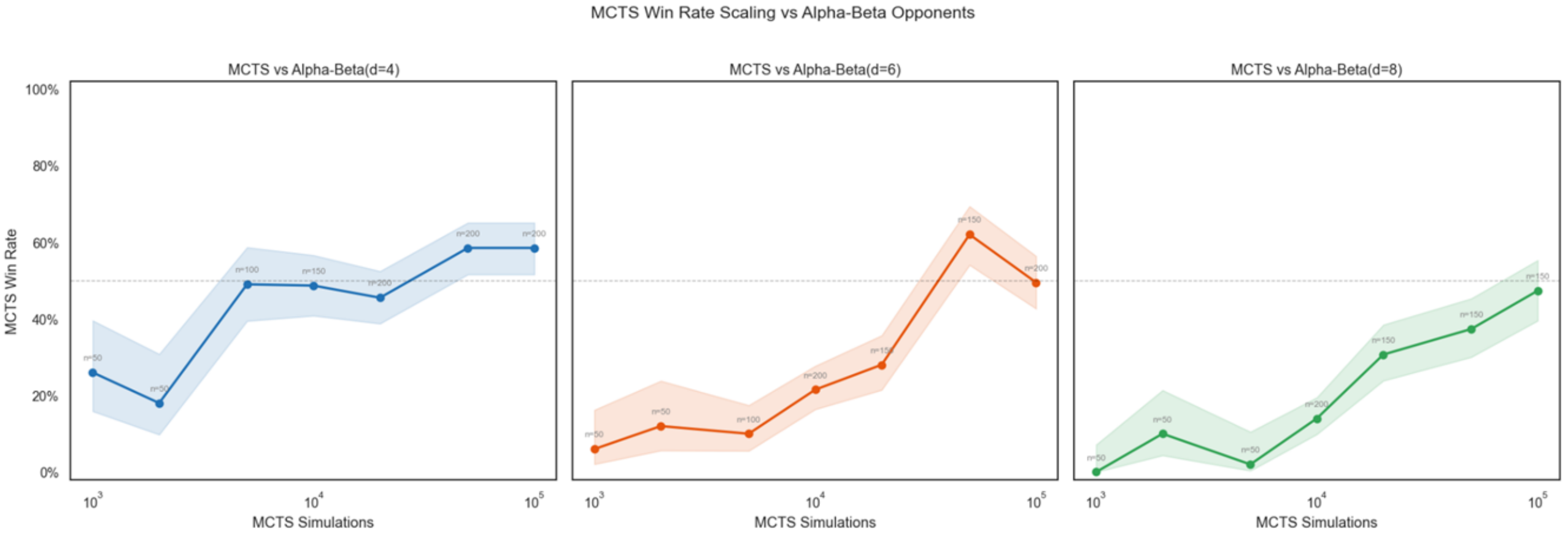
Evaluations – Efficiency

Agent	Elo	ms/move
MCTS(n=100000)	1724	12.237
Alpha-Beta(d=8)	1716	0.563
Alpha-Beta(d=6)	1694	0.065
MCTS(n=50000)	1669	5.940
Minimax(d=6)	1656	1.076
Alpha-Beta(d=4)	1574	0.006
MCTS(n=20000)	1569	2.103
Minimax(d=4)	1545	0.019
MCTS(n=10000)	1510	0.966
Alpha-Beta(d=7)	1500	0.305
Alpha-Beta(d=5)	1500	0.007
Alpha-Beta(d=3)	1500	0.001
MCTS(n=5000)	1410	0.537
MCTS(n=2000)	1323	0.313
MCTS(n=1000)	1220	0.138



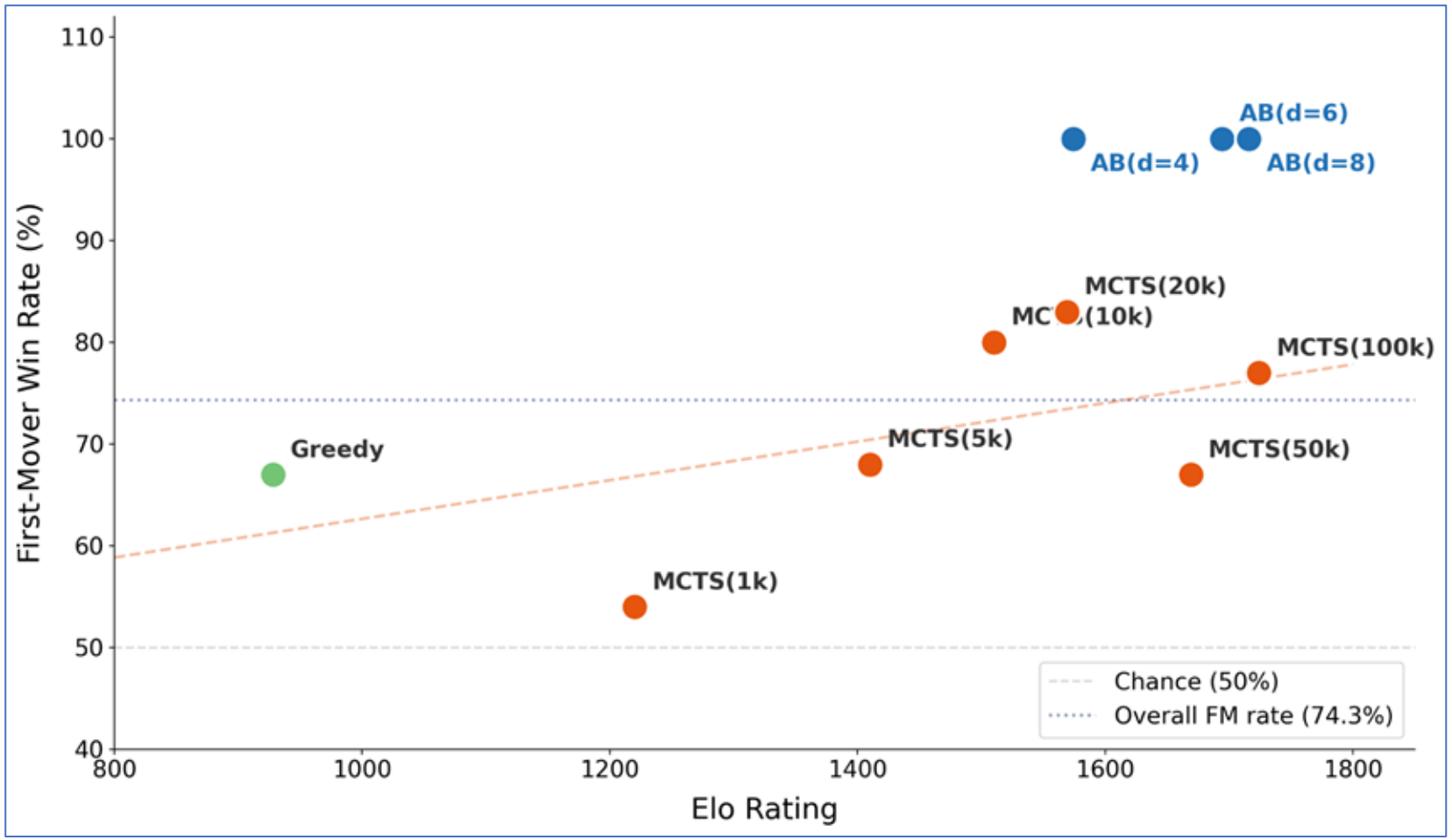
PennState

MCTS win rate vs. Alpha-Beta



PennState

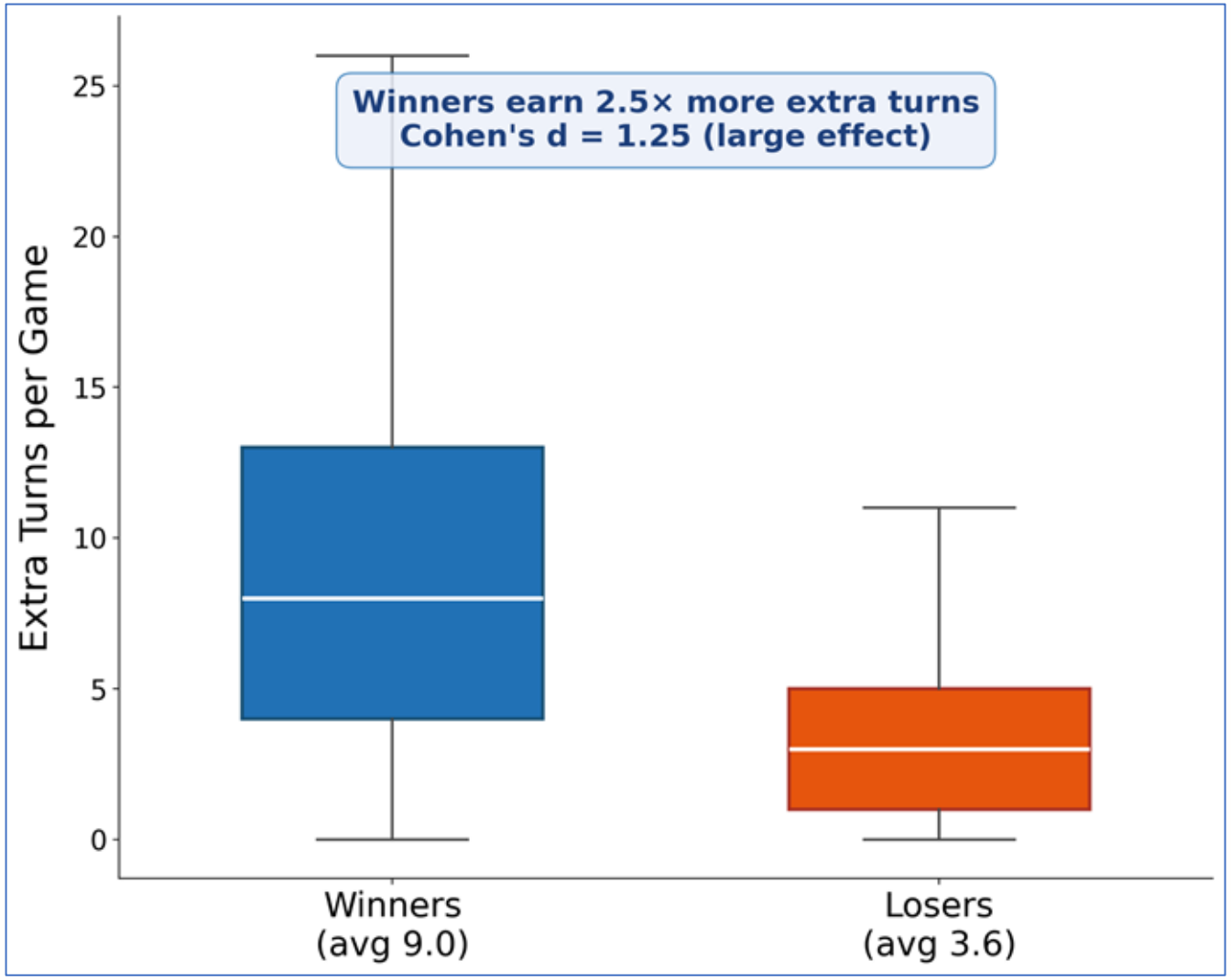
First Mover Advantage

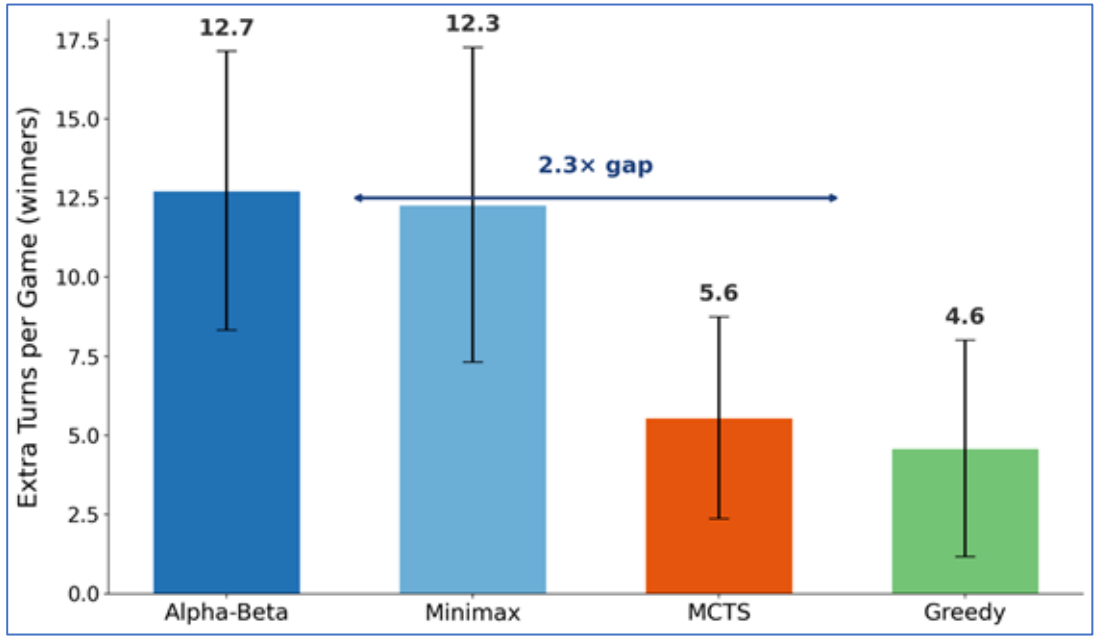


- FM win rate scales from 54% to 100% with agent strength
- Overall: 74.3% across 8,901 decided games

PennState

Tactical Drivers: Extra Turns





PennState

Discussion of Results

- Alpha-Beta Efficiency
  - Achieved 3x-16x speedup over Minimax with zero loss in decision quality
- Search Depth Ceiling
  - Performance peaked at depth of 6 nodes
    - Deeper searches only added computational cost
- MCTS vs. Alpha-Beta
  - MCTS required ~50,000 simulation to achieve parity with Alpha-Beta searching 6 nodes deep

PennState

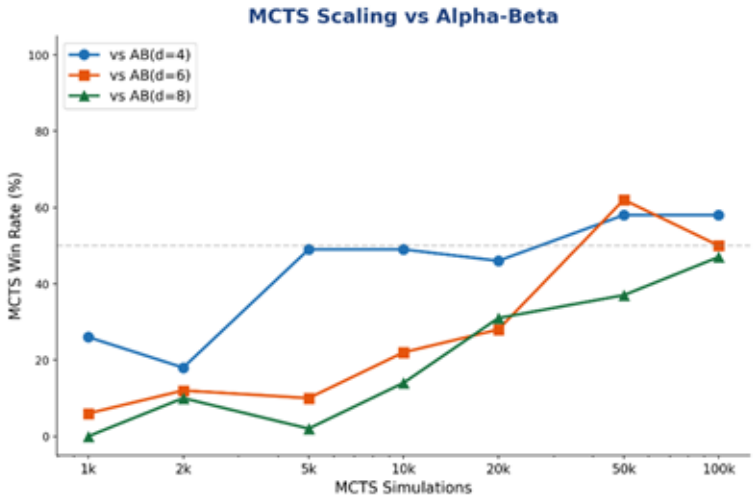
Discussion of Results

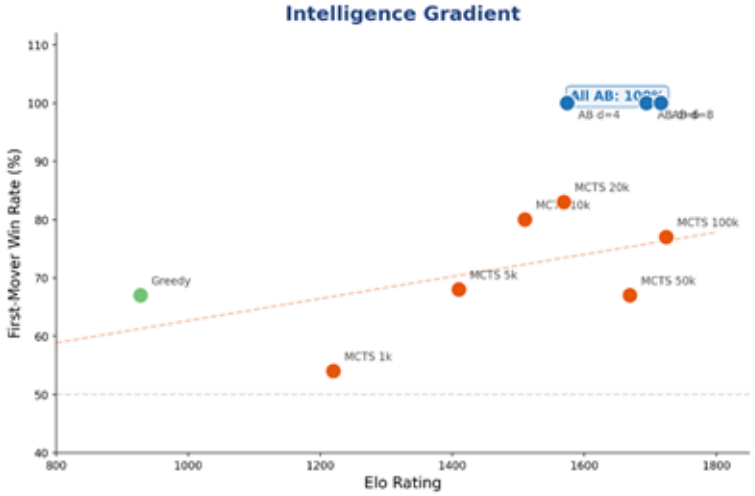
- Search vs. Evaluation
  - Increased search depth consistently outperformed evaluation function sophistication
- First-Mover Advantage
  - First mover won 74.3% of decided games, increasing with agent strength
- Tactical Drivers
  - Extra turns are the primary predictor of success
    - Winners earned about 2.5× as many extra turns as losers

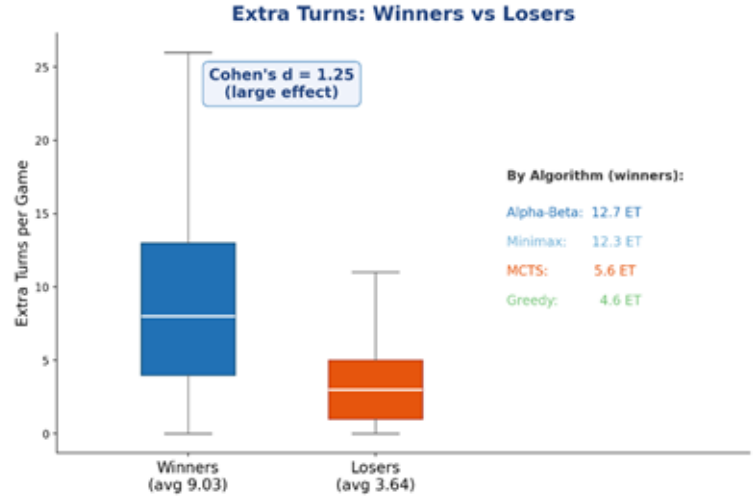
PennState

Dashboard Snippet









PennState

Conclusion

- Alpha-beta pruning at depth 6 is the strongest practical agent: top-tier strength at 0.068 ms per move
- Extra turns are the dominant tactical mechanism: winners earn 2.5x more than losers
- First-mover exploitation scales with agent intelligence: 54% to 100%
- Search depth matters more than evaluation sophistication

PennState

Limitations and Future Work

- MCTS used random rollouts only; heuristic-guided policies could close the gap with alpha-beta
- Single game variant (4-stone Kalah); generalizability untested
- Future:
  - Validate MCTS scaling predictions at higher budgets
  - Test heuristic-guided MCTS alternative
  - Test fair starting positions to isolate the intelligence gradient from structural first-mover advantage

PennState

Back Up slides

- References
- Model abstractions etc.
- Links

PennState

Minimax Pseudocode

```
1:  function maxAgent(board, player)
2:      bestValue = --infinity
3:      chosenMove = [ ]
4:      M = getAvailableMoves(board)
5:      pieces = store(board, player)
6:      for all m in M do
7:          if evaluate(m, board, player, pieces) ≥ bestValue then
8:              bestValue = evaluate(m, board, player, pieces)
9:              chosenMove += m
10:         end if
11:     end for
12:     return chosenMove
13: end function
14:
15: function evaluate(move, board, player, pieces)
16:     if doMove(board, move) is not terminal
17:         pieces = evaluate(move, board, player, pieces)
18:     else
19:         pieces = store(board, player)
20:     end if
21:     return pieces
22: end function
```



PennState

```
1:  function alphaBeta(node, α, β, player, opponent, depth)
2:      if node is terminal or depth = 0 then // leaf node
3:          return evaluate(board, player, opponent)
4:      end if
5:      if whoIsPlaying(board) = player // player playing: maximize
6:          bestValue = --infinity
7:          children[ ] = children(board)
8:          for all child in children do
9:              value = alphaBeta(child, α, β, callingPlayer, depth-1)
10:             bestValue = max(bestValue, val)
11:             α = max(α, bestValue)
12:             if β ≤ α then //pruning
13:                 break
14:             end if
15:         end for
16:         return bestValue
17:     else // opponent playing: minimize
18:         bestValue = infinity
19:         children[ ] = children(board)
20:         for all child in children do
21:             value = alphaBeta(child, α, β, callingPlayer, depth-1)
22:             bestValue = min(bestValue, val)
23:             β = min(β, bestValue)
24:             if β ≤ α then //pruning
25:                 break
26:             end if
27:         end for
28:         return bestValue
29:     end if
30: end function
31:
32: function evaluate(board, player, opponent)
33:     return store(board, player) - store(board, opponent)
34: end function
35:
36: function children(board)
37:     M = getAvailableMoves(board)
38:     for all m in M do
39:         child = doMove(board, m)
40:         add child to children
41:     end for
42:     return children
43: end function
```



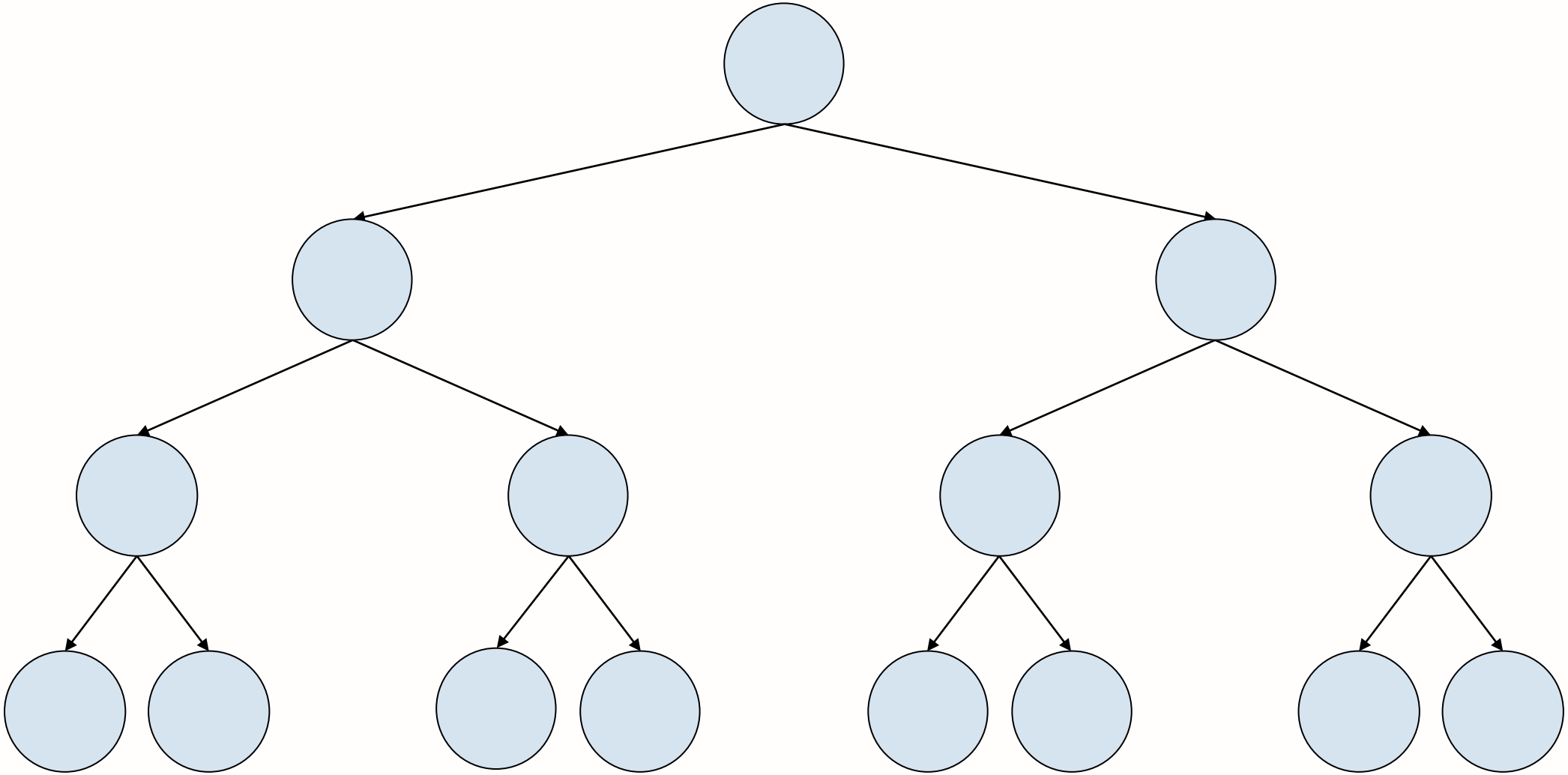
PennState

MCTS Psuedocode

```
1:  function mcts(state, N, C)
2:      rootPlayer ← state.toMove
3:      root ← new Node(state)
4:      for i = 1 to N do
5:          // Phase 1: Selection
6:          node ← root
7:          while node.children ≠ ∅ do
8:              node ← argmax_{c ∈ children} UCB(c, C)
9:          end while
10:         // Phase 2: Expansion
11:         if node is non-terminal and unexpanded then
12:             actions ← shuffle(legalMoves(node.state))
13:             for all action in actions do
14:                 child ← new Node(result(node.state, action))
15:                 add child to node.children
16:             end for
17:             node ← argmax_{c ∈ children} UCB(c, C)
18:         end if
19:         // Phase 3: Simulation (random rollout)
20:         simState ← node.state
21:         while simState is non-terminal do
22:             simState ← result(simState, randomChoice(legalMoves))
23:         end while
24:         utility ← gameUtility(simState, rootPlayer)
25:         // Phase 4: Backpropagation
26:         while node ≠ null do
27:             node.N ← node.N + 1
28:             parentPlayer ← node.parent.state.toMove
29:             u ← if parentPlayer = rootPlayer then utility else −utility
30:             node.U ← node.U + (u + 1) / 2  // maps {−1,0,+1} → {0,½,1}
31:             node ← node.parent
32:         end while
33:     end for
34:     return action of argmax_{c ∈ root.children} c.N
35: end function
```

```
36: function UCB(node, C)
37:     if node.N = 0 then
38:         return ∞
39:     end if
40:     return node.U / node.N + C · √(ln(node.parent.N) / node.N)
41: end function
```

PennState



PennState